

| РБНФ | Опис граматики за допомогою РБНФ | Опис граматики (правила)                                                                                                                                                                                                                                                   | LL2-нотація для правил граматикиОпис граматики | Код для перевірки РБНФ                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           | Код для перевірки граматики заданої за допомогою РБНФ | Дані для перевірки перевірки граматики за допомогою LL2-аналізатора<br>УВАГА: при копіюванні зважайте, щоб у кожному рядку після символу «\» не містилось жодних інших символів. |
|------|----------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|      |                                  | G = (N, T, P, S)                                                                                                                                                                                                                                                           |                                                |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |                                                       |                                                                                                                                                                                  |
|      |                                  | N = {<br>tokens_in_program,<br>token_iteration,<br>token,<br>keyword,<br>ident,<br>letter_in_lower_case,<br>letter_in_upper_case,<br>value,<br>sign_optional,<br>sign,<br>sign_plus,<br>sign_minus,<br>unsigned_value,<br>digit,<br>digit_optional,<br>non_zero_digit<br>} |                                                | N = {<br>tokens_in_program,<br>token_iteration,<br>token,<br>keyword,<br>ident,<br>letter_in_lower_case,<br>letter_in_upper_case,<br>value,<br>sign_optional,<br>sign,<br>sign_plus,<br>sign_minus,<br>unsigned_value,<br>digit,<br>digit_optional,<br>non_zero_digit<br>}                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |                                                       |                                                                                                                                                                                  |
|      |                                  |                                                                                                                                                                                                                                                                            |                                                | T = {<br>";" ,<br>"GOTO" ,<br>"INTEGER16" ,<br>" " ,<br>"," ,<br>"NOT" ,<br>"AND" ,<br>"OR" ,<br>"==" ,<br>"!=" ,<br>"<=" ,<br>">=" ,<br>"<" ,<br>">" ,<br>"+" ,<br>"-" ,<br>"*" ,<br>"/" ,<br>"DIV" ,<br>"MOD" ,<br>"(" ,<br>")" ,<br>"{" ,<br>"}" ,<br>"[" ,<br>"]" ,<br>"=" ,<br>"ELSE" ,<br>"IF" ,<br>"DO" ,<br>"FOR" ,<br>"TO" ,<br>"DOWNTO" ,<br>"WHILE" ,<br>"CONTINUE" ,<br>"BREAK" ,<br>"EXIT" ,<br>"REPEAT" ,<br>"UNTIL" ,<br>"GET" ,<br>"PUT" ,<br>"NAME" ,<br>"BODY" ,<br>"DATA" ,<br>"BEGIN" ,<br>"END" ,<br>"{" ,<br>"}" ,<br>"[" ,<br>"]" ,<br>";" ,<br>"-" ,<br>"+" ,<br>"0" ,<br>"1" ,<br>"2" ,<br>"3" ,<br>"4" ,<br>"5" ,<br>"6" ,<br>"7" ,<br>"8" ,<br>"9" ,<br>" " ,<br>"_" ,<br>"a" ,<br>"b" ,<br>"c" ,<br>"d" ,<br>"e" ,<br>"f" ,<br>"g" ,<br>"h" ,<br>"i" ,<br>"j" ,<br>"k" ,<br>"l" ,<br>"m" ,<br>"n" ,<br>"o" ,<br>"p" ,<br>"q" ,<br>"r" ,<br>"s" ,<br>"t" ,<br>"u" ,<br>"v" ,<br>"w" ,<br>"x" ,<br>"y" ,<br>"z" ,<br>} |                                                       |                                                                                                                                                                                  |

|  |  |  |  |                                                                                                                                                                                                                  |  |                                                                                                                       |
|--|--|--|--|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--|-----------------------------------------------------------------------------------------------------------------------|
|  |  |  |  | "A",<br>"B",<br>"C",<br>"D",<br>"E",<br>"F",<br>"G",<br>"H",<br>"I",<br>"J",<br>"K",<br>"L",<br>"M",<br>"N",<br>"O",<br>"P",<br>"Q",<br>"R",<br>"S",<br>"T",<br>"U",<br>"V",<br>"W",<br>"X",<br>"Y",<br>"Z"<br>} |  |                                                                                                                       |
|  |  |  |  |                                                                                                                                                                                                                  |  |                                                                                                                       |
|  |  |  |  | tokenCOLON = ":" >> BOUNDARIES;                                                                                                                                                                                  |  | #define T_COLON_0 ":"<br>#define T_COLON_1 ""<br>#define T_COLON_2 ""<br>#define T_COLON_3 ""                         |
|  |  |  |  | tokenGOTO = "GOTO" >> STRICT_BOUNDARIES;                                                                                                                                                                         |  | #define T_GOTO_0 "GOTO"<br>#define T_GOTO_1 ""<br>#define T_GOTO_2 ""<br>#define T_GOTO_3 ""                          |
|  |  |  |  | tokenINTEGER16 = "INTEGER16" >> STRICT_BOUNDARIES;                                                                                                                                                               |  | #define T_DATA_TYPE_0 "INTEGER16"<br>#define T_DATA_TYPE_1 ""<br>#define T_DATA_TYPE_2 ""<br>#define T_DATA_TYPE_3 "" |
|  |  |  |  | tokenCOMMA = "," >> BOUNDARIES;                                                                                                                                                                                  |  | #define T_COMA_0 ","<br>#define T_COMA_1 ""<br>#define T_COMA_2 ""<br>#define T_COMA_3 ""                             |
|  |  |  |  |                                                                                                                                                                                                                  |  | #define T_BITWISE_NOT_0 "~"<br>#define T_BITWISE_NOT_1 ""<br>#define T_BITWISE_NOT_2 ""<br>#define T_BITWISE_NOT_3 "" |
|  |  |  |  | tokenNOT = "NOT" >> STRICT_BOUNDARIES;                                                                                                                                                                           |  | #define T_NOT_0 "NOT"<br>#define T_NOT_1 ""<br>#define T_NOT_2 ""<br>#define T_NOT_3 ""                               |
|  |  |  |  |                                                                                                                                                                                                                  |  | #define T_BITWISE_AND_0 "&"<br>#define T_BITWISE_AND_1 ""<br>#define T_BITWISE_AND_2 ""<br>#define T_BITWISE_AND_3 "" |
|  |  |  |  | tokenAND = "AND" >> STRICT_BOUNDARIES;                                                                                                                                                                           |  | #define T_AND_0 "AND"<br>#define T_AND_1 ""<br>#define T_AND_2 ""<br>#define T_AND_3 ""                               |
|  |  |  |  |                                                                                                                                                                                                                  |  | #define T_BITWISE_OR_0 " "<br>#define T_BITWISE_OR_1 ""<br>#define T_BITWISE_OR_2 ""<br>#define T_BITWISE_OR_3 ""     |
|  |  |  |  | tokenOR = "OR" >> STRICT_BOUNDARIES;                                                                                                                                                                             |  | #define T_OR_0 "OR"<br>#define T_OR_1 ""<br>#define T_OR_2 ""<br>#define T_OR_3 ""                                    |
|  |  |  |  | tokenEQUAL = "==" >> BOUNDARIES;                                                                                                                                                                                 |  | #define T_EQUAL_0 "=="<br>#define T_EQUAL_1 ""<br>#define T_EQUAL_2 ""<br>#define T_EQUAL_3 ""                        |
|  |  |  |  | tokenNOTEQUAL = "!=" >> BOUNDARIES;                                                                                                                                                                              |  | #define T_NOT_EQUAL_0 "!="<br>#define T_NOT_EQUAL_1 ""<br>#define T_NOT_EQUAL_2 ""<br>#define T_NOT_EQUAL_3 ""        |
|  |  |  |  | tokenLESSOREQUAL = "<=" >> BOUNDARIES;                                                                                                                                                                           |  | #define T_LESS_OR_EQUAL_0 "<="                                                                                        |
|  |  |  |  |                                                                                                                                                                                                                  |  | #define T_LESS_OR_EQUAL_1 ""<br>#define T_LESS_OR_EQUAL_2 ""<br>#define T_LESS_OR_EQUAL_3 ""                          |
|  |  |  |  | tokenGREATEROREQUAL = ">=" >> BOUNDARIES;                                                                                                                                                                        |  | #define T_GREATER_OR_EQUAL_0 ">="                                                                                     |
|  |  |  |  |                                                                                                                                                                                                                  |  | #define T_GREATER_OR_EQUAL_1 ""<br>#define T_GREATER_OR_EQUAL_2 ""<br>#define T_GREATER_OR_EQUAL_3 ""                 |
|  |  |  |  | tokenLESS = "<" >> BOUNDARIES;                                                                                                                                                                                   |  | #define T_LESS_0 "<"<br>#define T_LESS_1 ""<br>#define T_LESS_2 ""<br>#define T_LESS_3 ""                             |
|  |  |  |  | tokenGREATER = ">" >> BOUNDARIES;                                                                                                                                                                                |  | #define T_GREATER_0 ">"<br>#define T_GREATER_1 ""<br>#define T_GREATER_2 ""<br>#define T_GREATER_3 ""                 |
|  |  |  |  | tokenPLUS = "+" >> BOUNDARIES;                                                                                                                                                                                   |  | #define T_ADD_0 "+"<br>#define T_ADD_1 ""<br>#define T_ADD_2 ""<br>#define T_ADD_3 ""                                 |
|  |  |  |  | tokenMINUS = "-" >> BOUNDARIES;                                                                                                                                                                                  |  | #define T_SUB_0 "-"<br>#define T_SUB_1 ""<br>#define T_SUB_2 ""<br>#define T_SUB_3 ""                                 |
|  |  |  |  | tokenMUL = "*" >> BOUNDARIES;                                                                                                                                                                                    |  | #define T_MUL_0 "*"<br>#define T_MUL_1 ""<br>#define T_MUL_2 ""<br>#define T_MUL_3 ""                                 |
|  |  |  |  | tokenDIV = "DIV" >> STRICT_BOUNDARIES;                                                                                                                                                                           |  | #define T_DIV_0 "DIV"<br>#define T_DIV_1 ""<br>#define T_DIV_2 ""                                                     |

|  |  |  |  |                                                  |  |                                                                                                                                                                |
|--|--|--|--|--------------------------------------------------|--|----------------------------------------------------------------------------------------------------------------------------------------------------------------|
|  |  |  |  |                                                  |  | #define T_DIV_3 ""                                                                                                                                             |
|  |  |  |  | tokenMOD = "MOD" >> STRICT_BOUNDARIES;           |  | #define T_MOD_0 "MOD"<br>#define T_MOD_1 ""<br>#define T_MOD_2 ""<br>#define T_MOD_3 ""                                                                        |
|  |  |  |  | tokenGROUPEXPRESSIONBEGIN = "{ " >> BOUNDARIES;  |  | #define T_BEGIN_GROUPEXPRESSION_0 "{ "<br>#define T_BEGIN_GROUPEXPRESSION_1 ""<br>#define T_BEGIN_GROUPEXPRESSION_2 ""<br>#define T_BEGIN_GROUPEXPRESSION_3 "" |
|  |  |  |  | tokenGROUPEXPRESSIONEND = "}" >> BOUNDARIES;     |  | #define T_END_GROUPEXPRESSION_0 "}"<br>#define T_END_GROUPEXPRESSION_1 ""<br>#define T_END_GROUPEXPRESSION_2 ""<br>#define T_END_GROUPEXPRESSION_3 ""          |
|  |  |  |  | tokenLRBIND = "=: " >> BOUNDARIES;               |  | #define T_LRBIND_0 "=: "<br>#define T_LRBIND_1 ""<br>#define T_LRBIND_2 ""<br>#define T_LRBIND_3 ""                                                            |
|  |  |  |  |                                                  |  | #define T_THEN_BLOCK_0 "{ "<br>#define T_THEN_BLOCK_1 ""<br>#define T_THEN_BLOCK_2 ""<br>#define T_THEN_BLOCK_3 ""                                             |
|  |  |  |  | tokenELSE = "ELSE" >> STRICT_BOUNDARIES;         |  | #define T_ELSE_BLOCK_0 "ELSE"<br>#define T_ELSE_BLOCK_1 "{ "<br>#define T_ELSE_BLOCK_2 ""<br>#define T_ELSE_BLOCK_3 ""                                         |
|  |  |  |  | tokenIF = "IF" >> STRICT_BOUNDARIES;             |  | #define T_IF_0 "IF"<br>#define T_IF_1 ""<br>#define T_IF_2 ""<br>#define T_IF_3 ""                                                                             |
|  |  |  |  |                                                  |  | #define T_ELSE_IF_0 "ELSE"<br>#define T_ELSE_IF_1 "IF"<br>#define T_ELSE_IF_2 ""<br>#define T_ELSE_IF_3 ""                                                     |
|  |  |  |  | tokenDO = "DO" >> STRICT_BOUNDARIES;             |  | #define T_DO_0 "DO"<br>#define T_DO_1 ""<br>#define T_DO_2 ""<br>#define T_DO_3 ""                                                                             |
|  |  |  |  | tokenFOR = "FOR" >> STRICT_BOUNDARIES;           |  | #define T_FOR_0 "FOR"<br>#define T_FOR_1 ""<br>#define T_FOR_2 ""<br>#define T_FOR_3 ""                                                                        |
|  |  |  |  | tokenTO = "TO" >> STRICT_BOUNDARIES;             |  | #define T_TO_0 "TO"<br>#define T_TO_1 ""<br>#define T_TO_2 ""<br>#define T_TO_3 ""                                                                             |
|  |  |  |  | tokenDOWNTO = "DOWNTO" >> STRICT_BOUNDARIES;     |  | #define T_DOWNTO_0 "DOWNTO"<br>#define T_DOWNTO_1 ""<br>#define T_DOWNTO_2 ""<br>#define T_DOWNTO_3 ""                                                         |
|  |  |  |  | tokenWHILE = "WHILE" >> STRICT_BOUNDARIES;       |  | #define T_WHILE_0 "WHILE"<br>#define T_WHILE_1 ""<br>#define T_WHILE_2 ""<br>#define T_WHILE_3 ""                                                              |
|  |  |  |  | tokenCONTINUE = "CONTINUE" >> STRICT_BOUNDARIES; |  | #define T_CONTINUE_WHILE_0 "CONTINUE"<br>#define T_CONTINUE_WHILE_1 ""<br>#define T_CONTINUE_WHILE_2 ""<br>#define T_CONTINUE_WHILE_3 ""                       |
|  |  |  |  | tokenBREAK = "BREAK" >> STRICT_BOUNDARIES;       |  | #define T_EXIT_WHILE_0 "BREAK"<br>#define T_EXIT_WHILE_1 ""<br>#define T_EXIT_WHILE_2 ""<br>#define T_EXIT_WHILE_3 ""                                          |
|  |  |  |  | tokenEXIT = "EXIT" >> STRICT_BOUNDARIES;         |  | #define T_EXIT_0 "EXIT"<br>#define T_EXIT_1 ""<br>#define T_EXIT_2 ""<br>#define T_EXIT_3 ""                                                                   |
|  |  |  |  | tokenREPEAT = "REPEAT" >> STRICT_BOUNDARIES;     |  | #define T_REPEAT_0 "REPEAT"<br>#define T_REPEAT_1 ""<br>#define T_REPEAT_2 ""<br>#define T_REPEAT_3 ""                                                         |
|  |  |  |  | tokenUNTIL = "UNTIL" >> STRICT_BOUNDARIES;       |  | #define T_UNTIL_0 "UNTIL"<br>#define T_UNTIL_1 ""<br>#define T_UNTIL_2 ""<br>#define T_UNTIL_3 ""                                                              |
|  |  |  |  | tokenGET = "GET" >> STRICT_BOUNDARIES;           |  | #define T_INPUT_0 "GET"<br>#define T_INPUT_1 ""<br>#define T_INPUT_2 ""<br>#define T_INPUT_3 ""                                                                |
|  |  |  |  | tokenPUT = "PUT" >> STRICT_BOUNDARIES;           |  | #define T_OUTPUT_0 "PUT"<br>#define T_OUTPUT_1 ""<br>#define T_OUTPUT_2 ""<br>#define T_OUTPUT_3 ""                                                            |
|  |  |  |  | tokenNAME = "NAME" >> STRICT_BOUNDARIES;         |  | #define T_NAME_0 "NAME"<br>#define T_NAME_1 ""<br>#define T_NAME_2 ""<br>#define T_NAME_3 ""                                                                   |
|  |  |  |  | tokenBODY = "BODY" >> STRICT_BOUNDARIES;         |  | #define T_BODY_0 "BODY"<br>#define T_BODY_1 ""<br>#define T_BODY_2 ""<br>#define T_BODY_3 ""                                                                   |
|  |  |  |  | tokenDATA = "DATA" >> STRICT_BOUNDARIES;         |  | #define T_DATA_0 "DATA"<br>#define T_DATA_1 ""<br>#define T_DATA_2 ""<br>#define T_DATA_3 ""                                                                   |
|  |  |  |  | tokenBEGIN = "BEGIN" >> STRICT_BOUNDARIES;       |  | #define T_BEGIN_0 "BEGIN"<br>#define T_BEGIN_1 ""<br>#define T_BEGIN_2 ""<br>#define T_BEGIN_3 ""                                                              |
|  |  |  |  | tokenEND = "END" >> STRICT_BOUNDARIES;           |  | #define T_END_0 "END"<br>#define T_END_1 ""<br>#define T_END_2 ""<br>#define T_END_3 ""                                                                        |
|  |  |  |  | tokenBEGINBLOCK = "{ " >> BOUNDARIES;            |  | #define T_BEGIN_BLOCK_0 "{ "<br>#define T_BEGIN_BLOCK_1 ""<br>#define T_BEGIN_BLOCK_2 ""<br>#define T_BEGIN_BLOCK_3 ""                                         |
|  |  |  |  | tokenENDBLOCK = "}" >> BOUNDARIES;               |  | #define T_END_BLOCK_0 "}"                                                                                                                                      |

|                                                                                                           |  |  |  |                                                                                                                                                                                                  |  |                                                                                                                                                           |
|-----------------------------------------------------------------------------------------------------------|--|--|--|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--|-----------------------------------------------------------------------------------------------------------------------------------------------------------|
|                                                                                                           |  |  |  |                                                                                                                                                                                                  |  | #define T_END_BLOCK_1 ""<br>#define T_END_BLOCK_2 ""<br>#define T_END_BLOCK_3 ""                                                                          |
|                                                                                                           |  |  |  | tokenLEFTSQUAREBRACKETS = "[" >> BOUNDARIES;                                                                                                                                                     |  | #define T_LEFT_SQUAREBRACKETS_0 "["<br>#define T_LEFT_SQUAREBRACKETS_1 ""<br>#define T_LEFT_SQUAREBRACKETS_2 ""<br>#define T_LEFT_SQUAREBRACKETS_3 ""     |
|                                                                                                           |  |  |  | tokenRIGHTSQUAREBRACKETS = "]" >> BOUNDARIES;                                                                                                                                                    |  | #define T_RIGHT_SQUAREBRACKETS_0 "]"<br>#define T_RIGHT_SQUAREBRACKETS_1 ""<br>#define T_RIGHT_SQUAREBRACKETS_2 ""<br>#define T_RIGHT_SQUAREBRACKETS_3 "" |
|                                                                                                           |  |  |  | tokenSEMICOLON = ";" >> BOUNDARIES;                                                                                                                                                              |  | #define T_SEMICOLON_0 ";"<br>#define T_SEMICOLON_1 ""<br>#define T_SEMICOLON_2 ""<br>#define T_SEMICOLON_3 ""                                             |
|                                                                                                           |  |  |  |                                                                                                                                                                                                  |  | #define T_NULL_STATEMENT_0 "NULL"<br>#define T_NULL_STATEMENT_1 "STATEMENT"<br>#define T_NULL_STATEMENT_2 ""<br>#define T_NULL_STATEMENT_3 ""             |
|                                                                                                           |  |  |  |                                                                                                                                                                                                  |  | #define GRAMMAR_LL2__2025 {\                                                                                                                              |
| labeled_point = ident , ":";                                                                              |  |  |  | labeled_point = ident >> tokenCOLON;                                                                                                                                                             |  | { LA_IS, {"ident_terminal"}, {"labeled_point"}, {\                                                                                                        |
| goto_label = "GOTO" , ident;                                                                              |  |  |  | goto_label = tokenGOTO >> ident;                                                                                                                                                                 |  | { LA_IS, {T_GOTO_0}, {"goto_label"}, {\                                                                                                                   |
| program_name = ident;                                                                                     |  |  |  | program_name = SAME_RULE(ident);                                                                                                                                                                 |  | { LA_IS, {"ident_terminal"}, {"program_name"}, {\                                                                                                         |
| value_type = "INTEGER16";                                                                                 |  |  |  | value_type = SAME_RULE(tokenINTEGER16);                                                                                                                                                          |  | { LA_IS, {T_DATA_TYPE_0}, {"value_type"}, {\                                                                                                              |
|                                                                                                           |  |  |  |                                                                                                                                                                                                  |  | { LA_IS, {"["}, {"array_specify"}, {\                                                                                                                     |
| declaration_element = ident , [ "[" , unsigned_value , "]" ];                                             |  |  |  | declaration_element = ident >> -(tokenLEFTSQUAREBRACKETS >> unsigned_value >> tokenRIGHTSQUAREBRACKETS);                                                                                         |  | { LA_IS, {"ident_terminal"}, {"declaration_element"}, {\                                                                                                  |
|                                                                                                           |  |  |  |                                                                                                                                                                                                  |  | { LA_IS, {"["}, {"array_specify_optional"}, {\                                                                                                            |
| other_declaration_ident = " , " , declaration_element;                                                    |  |  |  | other_declaration_ident = tokenCOMMA >> declaration_element;                                                                                                                                     |  | { LA_IS, {T_COMA_0}, {"other_declaration_ident"}, {\                                                                                                      |
| declaration = value_type , declaration_element , {other_declaration_ident};                               |  |  |  | declaration = value_type >> declaration_element >> *other_declaration_ident;                                                                                                                     |  | { LA_IS, {T_DATA_TYPE_0}, {"declaration"}, {\                                                                                                             |
|                                                                                                           |  |  |  |                                                                                                                                                                                                  |  | { LA_IS, {T_COMA_0}, {"other_declaration_ident__iteration"}, {\                                                                                           |
| index_action = "[" , expression , "]" ;                                                                   |  |  |  | index_action = tokenLEFTSQUAREBRACKETS >> expression >> tokenRIGHTSQUAREBRACKETS;                                                                                                                |  | { LA_IS, {"["}, {"index_action"}, {\                                                                                                                      |
| unary_operator = "NOT";                                                                                   |  |  |  | unary_operator = SAME_RULE(tokenNOT);                                                                                                                                                            |  | { LA_IS, {T_NOT_0}, {"unary_operator"}, {\                                                                                                                |
| unary_operation = unary_operator , expression;                                                            |  |  |  | unary_operation = unary_operator >> expression;                                                                                                                                                  |  | { LA_IS, {T_NOT_0}, {"unary_operation"}, {\                                                                                                               |
| binary_operator = "AND"   "OR"   "=="   "!="   "<="   ">="   "<"   ">"   "+"   "-"   "*"   "DIV"   "MOD"; |  |  |  | binary_operator = tokenAND   tokenOR   tokenEQUAL   tokenNOTEQUAL   tokenLESSOREQUAL   tokenGREATEROREQUAL   tokenLESS   tokenGREATER   tokenPLUS   tokenMINUS   tokenMUL   tokenDIV   tokenMOD; |  | { LA_IS, {T_AND_0}, {"binary_operator"}, {\                                                                                                               |

|                                                                                                         |  |  |  |                                                                                                              |  |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |
|---------------------------------------------------------------------------------------------------------|--|--|--|--------------------------------------------------------------------------------------------------------------|--|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                                                                                                         |  |  |  |                                                                                                              |  | { LA_IS, { T_MOD_0 }, { "binary_operator", {\ { LA_IS, { "" }, 1, { T_MOD_0 } } }\ }}}\                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |
| binary_action = binary_operator , expression;                                                           |  |  |  | binary_action = binary_operator >> expression;                                                               |  | IF_NONUSE_COMPARE_WITH_EQUAL(\ { LA_IS, { T_AND_0, T_OR_0, T_EQUAL_0, T_NOT_EQUAL_0, T_LESS_0, T_GREATER_0, T_ADD_0, T_SUB_0, T_MUL_0, T_DIV_0, T_MOD_0 }, {\ "binary_action", {\ { LA_IS, { "" }, 2, { "binary_operator", "expression" } } }\ }}}\ )\ IF_USE_COMPARE_WITH_EQUAL(\ { LA_IS, { T_AND_0, T_OR_0, T_EQUAL_0, T_NOT_EQUAL_0, T_LESS_OR_EQUAL_0, T_GREATER_OR_EQUAL_0, T_ADD_0, T_SUB_0, T_MUL_0, T_DIV_0, T_MOD_0 }, {\ "binary_action", {\ { LA_IS, { "" }, 2, { "binary_operator", "expression" } } }\ }}}\ )\                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
| left_expression = group_expression   unary_operation   ident , [index_action]   value   cond_block;     |  |  |  | left_expression = group_expression   unary_operation   ident >> -index_action   value   cond_block;          |  | {LA_IS, { "[" }, { "left_expression", {\ {LA_IS, { "" }, 1, { "group_expression" } } }\ }}}\ {LA_IS, { T_NOT_0 }, { "left_expression", {\ {LA_IS, { "" }, 1, { "unary_operation" } } }\ }}}\ {LA_IS, { T_ADD_0, T_SUB_0 }, { "left_expression", {\ {LA_IS, { "unsigned_value_terminal", 1, { "value" } } },\ {LA_NOT, { "unsigned_value_terminal", 1, { "unary_operation" } } }\ }}}\ {LA_IS, { "ident_terminal" }, { "left_expression", {\ {LA_IS, { "" }, 2, { "ident", "index_action__optional" } } }\ }}}\ {LA_IS, { "unsigned_value_terminal", 1, { "value" } } }, {\ {LA_IS, { "" }, 1, { "value" } } }\ }}}\ {LA_IS, { "T_IF_0" }, { "left_expression", {\ {LA_IS, { "" }, 1, { "cond_block" } } }\ }}}\                                                                                                                                                                                                                                                                                    |
|                                                                                                         |  |  |  |                                                                                                              |  | {LA_IS, { "[" }, { "index_action__optional", {\ {LA_IS, { "" }, 1, { "index_action" } } }\ }}}\ {LA_NOT, { "[" }, { "index_action__optional", {\ {LA_IS, { "" }, 0, { "" } } }\ }}}\                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |
| expression = left_expression , [binary_action];                                                         |  |  |  | expression = left_expression >> *binary_action;                                                              |  | {LA_IS, { "(", T_NOT_0, T_ADD_0, T_SUB_0, "ident_terminal", "unsigned_value_terminal", "T_IF_0" }, {\ "expression", {\ {LA_IS, { "" }, 2, { "left_expression", "binary_action__iteration" } } }\ }}}\                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
|                                                                                                         |  |  |  |                                                                                                              |  | IF_NONUSE_COMPARE_WITH_EQUAL(\ { LA_IS, { T_AND_0, T_OR_0, T_EQUAL_0, T_NOT_EQUAL_0, T_LESS_0, T_GREATER_0, T_ADD_0, T_SUB_0, T_MUL_0, T_DIV_0, T_MOD_0 }, {\ "binary_action__iteration", {\ { LA_IS, { "" }, 2, { "binary_action", "binary_action__iteration" } } }\ }}}\ {LA_NOT, { T_AND_0, T_OR_0, T_EQUAL_0, T_NOT_EQUAL_0, T_LESS_0, T_GREATER_0, T_ADD_0, T_SUB_0, T_MUL_0, T_DIV_0, T_MOD_0 }, {\ "binary_action__iteration", {\ { LA_IS, { "" }, 0, { "" } } }\ }}}\ )\ IF_USE_COMPARE_WITH_EQUAL(\ { LA_IS, { T_AND_0, T_OR_0, T_EQUAL_0, T_NOT_EQUAL_0, T_LESS_OR_EQUAL_0, T_GREATER_OR_EQUAL_0, T_ADD_0, T_SUB_0, T_MUL_0, T_DIV_0, T_MOD_0 }, {\ "binary_action__iteration", {\ { LA_IS, { "" }, 2, { "binary_action", "binary_action__iteration" } } }\ }}}\ {LA_NOT, { T_AND_0, T_OR_0, T_EQUAL_0, T_NOT_EQUAL_0, T_LESS_OR_EQUAL_0, T_GREATER_OR_EQUAL_0, T_ADD_0, T_SUB_0, T_MUL_0, T_DIV_0, T_MOD_0 }, {\ "binary_action__iteration", {\ { LA_IS, { "" }, 0, { "" } } }\ }}}\ )\ |
| group_expression = "(" , expression , ")";                                                              |  |  |  | group_expression = tokenGROUPEXPRESSIONBEGIN >> expression >> tokenGROUPEXPRESSIONEND;                       |  | {LA_IS, { "(" }, { "group_expression", {\ {LA_IS, { "" }, 3, { "(", "expression", ")" } } }\ }}}\                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |
| expression_or_bind_left_to_right = expression , ["=" , ident , [index_action]];                         |  |  |  | expression_or_bind_left_to_right = expression >> -(tokenLRBIND >> ident >> -index_action);                   |  | {LA_IS, { "(", T_NOT_0, T_ADD_0, T_SUB_0, "ident_terminal", "unsigned_value_terminal", "T_IF_0" }, {\ "expression_or_bind_left_to_right", {\ {LA_IS, { "" }, 2, { "expression", "bind_to_right__optional" } } }\ }}}\                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
|                                                                                                         |  |  |  |                                                                                                              |  | {LA_IS, { T_LRBIND_0 }, { "bind_to_right", {\ {LA_IS, { "" }, 3, { T_LRBIND_0, "ident", "index_action__optional" } } }\ }}}\                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
|                                                                                                         |  |  |  |                                                                                                              |  | { LA_IS, { T_LRBIND_0 }, { "bind_to_right__optional", {\ { LA_IS, { "" }, 1, { "bind_to_right" } } }\ }}}\ { LA_NOT, { T_LRBIND_0 }, { "bind_to_right__optional", {\ { LA_IS, { "" }, 0, { "" } } }\ }}}\                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |
| if_expression = expression;                                                                             |  |  |  | if_expression = SAME_RULE(expression);                                                                       |  | {LA_IS, { "(", T_NOT_0, T_ADD_0, T_SUB_0, "ident_terminal", "unsigned_value_terminal", "T_IF_0" }, {\ "if_expression", {\ {LA_IS, { "" }, 1, { "expression" } } }\ }}}\                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |
| body_for_true = block_statements_in_while_and_if_body;                                                  |  |  |  | body_for_true = SAME_RULE(block_statements_in_while_and_if_body);                                            |  | {LA_IS, { T_BEGIN_BLOCK_0 }, {\ "body_for_true", {\ {LA_IS, { "" }, 1, { "block_statements_in_while_and_if_body" } } }\ }}}\                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
| false_cond_block_without_else = "ELSE" , "IF" , if_expression , body_for_true;                          |  |  |  | false_cond_block_without_else = tokenELSE >> tokenIF >> if_expression >> body_for_true;                      |  | {LA_IS, { T_ELSE_IF_0 }, {\ "false_cond_block_without_else", {\ {LA_IS, { "" }, 4, { T_ELSE_IF_0, T_ELSE_IF_1, "if_expression", "body_for_true" } } }\ }}}\                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
| body_for_false = "ELSE" , block_statements_in_while_and_if_body;                                        |  |  |  | body_for_false = tokenELSE >> block_statements_in_while_and_if_body;                                         |  | {LA_IS, { T_ELSE_BLOCK_0 }, {\ "body_for_false", {\ {LA_IS, { "" }, 2, { T_ELSE_BLOCK_0, "block_statements" } } }\ }}}\                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |
| cond_block = "IF" , if_expression , body_for_true, {false_cond_block_without_else } , [body_for_false]; |  |  |  | cond_block = tokenIF >> if_expression >> body_for_true >> *false_cond_block_without_else >> -body_for_false; |  | {LA_IS, { T_IF_0 }, {\ "cond_block", {\ {LA_IS, { "" }, 5, { T_IF_0, "if_expression", "body_for_true", "false_cond_block_without_else__iteration", "body_for_false__optional" } } }\ }}}\                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |
|                                                                                                         |  |  |  |                                                                                                              |  | {LA_IS, { T_ELSE_IF_0 }, {\ "false_cond_block_without_else__iteration", {\ {LA_IS, { T_ELSE_IF_1 }, 2, { "false_cond_block_without_else", "false_cond_block_without_else__iteration" } } }\                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |

|                                                                                                       |  |  |  |                                                                                                                    |  |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
|-------------------------------------------------------------------------------------------------------|--|--|--|--------------------------------------------------------------------------------------------------------------------|--|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                                                                                                       |  |  |  |                                                                                                                    |  | {LA_NOT, { T_ELSE_IF_1 }, 0, { "" }}\ }};\ {LA_NOT, { T_ELSE_IF_0 }, { "false_cond_block_without_else__iteration",{\ {LA_IS, {""}, 0, { "" }}\ }};\ }};\                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
|                                                                                                       |  |  |  |                                                                                                                    |  | {LA_IS, {T_ELSE_BLOCK_0 }, { "body_for_false__optional",{\ {LA_IS, {""}, 1, { "body_for_false" }}\ }};\ {LA_NOT, { T_ELSE_BLOCK_0 }, { "body_for_false__optional",{\ {LA_IS, {""}, 0, { "" }}\ }};\ }};\                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
| cond_block__with_optional_bind = cond_block, [tokenLRBIND , ident , {index_action}];                  |  |  |  | cond_block__with_optional_bind = cond_block >> -(tokenLRBIND >> ident >> -index_action);                           |  | {LA_IS, { T_IF_0 }, { "cond_block__with_optional_bind",{\ {LA_IS, {""}, 2, { "cond_block", "bind_to_right__optional" }}\ }};\ }};\                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
| cycle_begin_expression = expression;                                                                  |  |  |  | cycle_begin_expression = SAME_RULE(expression);                                                                    |  | {LA_IS, { "(", T_NOT_0, T_ADD_0, T_SUB_0, "ident_terminal", "unsigned_value_terminal", "T_IF_0" }, { "cycle_begin_expression",{\ {LA_IS, {""}, 1, { "expression" }}\ }};\ }};\                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |
| cycle_end_expression = expression;                                                                    |  |  |  | cycle_end_expression = SAME_RULE(expression);                                                                      |  | {LA_IS, { "(", T_NOT_0, T_ADD_0, T_SUB_0, "ident_terminal", "unsigned_value_terminal", "T_IF_0" }, { "cycle_end_expression",{\ {LA_IS, {""}, 1, { "expression" }}\ }};\ }};\                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |
| cycle_counter = ident;                                                                                |  |  |  | cycle_counter = SAME_RULE(ident);                                                                                  |  | {LA_IS, { "ident_terminal" }, { "cycle_counter",{\ {LA_IS, {""}, 1, { "ident" }}\ }};\ }};\                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |
| cycle_counter_lr_init = cycle_begin_expression , "=: " , cycle_counter;                               |  |  |  | cycle_counter_lr_init = cycle_begin_expression >> tokenLRBIND >> cycle_counter;                                    |  | {LA_IS, { "(", T_NOT_0, T_ADD_0, T_SUB_0, "ident_terminal", "unsigned_value_terminal", "T_IF_0" }, { "cycle_counter_lr_init",{\ {LA_IS, {""}, 3, { "cycle_begin_expression", T_LRBIND_0, "cycle_counter" }}\ }};\ }};\                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |
| cycle_counter_init = cycle_counter_lr_init;                                                           |  |  |  | cycle_counter_init = SAME_RULE(cycle_counter_lr_init);                                                             |  | {LA_IS, { "(", T_NOT_0, T_ADD_0, T_SUB_0, "ident_terminal", "unsigned_value_terminal", "T_IF_0" }, { "cycle_counter_init",{\ {LA_IS, {""}, 1, { "cycle_counter_lr_init" }}\ }};\ }};\                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |
| cycle_counter_last_value = cycle_end_expression;                                                      |  |  |  | cycle_counter_last_value = SAME_RULE(cycle_end_expression);                                                        |  | {LA_IS, { "(", T_NOT_0, T_ADD_0, T_SUB_0, "ident_terminal", "unsigned_value_terminal", "T_IF_0" }, { "cycle_counter_last_value",{\ {LA_IS, {""}, 1, { "cycle_end_expression" }}\ }};\ }};\                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |
| cycle_body = "DO" , ({statement}   block_statements);                                                 |  |  |  | cycle_body = tokenDO >> (statement   block_statements);                                                            |  | {LA_IS, { T_DO_0 }, { "cycle_body",{\ {LA_IS, {""}, 2, { T_DO_0, "statements__or__block_statements" }}\ }};\ }};\                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |
|                                                                                                       |  |  |  |                                                                                                                    |  | {LA_IS, { T_TO_0 }, { "forto_direction",{\ {LA_IS, {""}, 1, { T_TO_0 }}\ }};\ }};\ {LA_IS, { T_DOWNT0_0 }, { "forto_direction",{\ {LA_IS, {""}, 1, { T_DOWNT0_0 }}\ }};\ }};\                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |
| forto_cycle = "FOR" , cycle_counter_init , ("TO"   "DOWNT0") , cycle_counter_last_value , cycle_body; |  |  |  | forto_cycle = tokenFOR >> cycle_counter_init >> (tokenTO   tokenDOWNT0) >> cycle_counter_last_value >> cycle_body; |  | {LA_IS, { T_FOR_0 }, { "forto_cycle",{\ {LA_IS, {""}, 5, { T_FOR_0, "cycle_counter_init", "forto_direction", "cycle_counter_last_value", "cycle_body" }}\ }};\ }};\                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |
|                                                                                                       |  |  |  | continue_while = SAME_RULE(tokenCONTINUE);                                                                         |  | {LA_IS, { "T_CONTINUE_WHILE_0" }, { "continue_while",{\ {LA_IS, {""}, 1, { "T_CONTINUE_WHILE_0" }}\ }};\ }};\                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |
|                                                                                                       |  |  |  | break_while = SAME_RULE(tokenBREAK);                                                                               |  | {LA_IS, { "T_EXIT_WHILE_0" }, { "break_while",{\ {LA_IS, {""}, 1, { "T_EXIT_WHILE_0" }}\ }};\ }};\                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
| statement_in_while_and_if_body = statement   "CONTINUE"   "BREAK";                                    |  |  |  | statement_in_while_and_if_body = statement   continue_while   break_while;                                         |  | {LA_IS, { "ident_terminal", "(", T_NOT_0, "unsigned_value_terminal", T_ADD_0, T_SUB_0, T_IF_0, T_FOR_0, T_WHILE_0, T_REPEAT_0, T_GOTO_0, T_INPUT_0, T_OUTPUT_0, T_SEMICOLON_0 }, { "statement_in_while_and_if_body",{\ {LA_IS, {""}, 1, { "statement" }}\ }};\ }};\ {LA_IS, { T_CONTINUE_WHILE_0 }, { "statement_in_while_and_if_body",{\ {LA_IS, {""}, 1, { "continue_while" }}\ }};\ }};\ {LA_IS, { T_EXIT_WHILE_0 }, { "statement_in_while_and_if_body",{\ {LA_IS, {""}, 1, { "break_while" }}\ }};\ }};\                                                                                                                                                                                    |
| block_statements_in_while_and_if_body = "{", {statement_in_while_and_if_body} , "}";                  |  |  |  | block_statements_in_while_and_if_body = tokenBEGINBLOCK >> *statement_in_while_and_if_body >> tokenENDBLOCK;       |  | {LA_IS, { T_BEGIN_BLOCK_0 }, { "block_statements_in_while_and_if_body",{\ {LA_IS, {""}, 3, { T_BEGIN_BLOCK_0, "statement_in_while_and_if_body__iteration", T_END_BLOCK_0 }}\ }};\ }};\                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |
|                                                                                                       |  |  |  |                                                                                                                    |  | {LA_IS, { "ident_terminal", "(", T_NOT_0, "unsigned_value_terminal", T_ADD_0, T_SUB_0, T_IF_0, T_FOR_0, T_WHILE_0, T_REPEAT_0, T_GOTO_0, T_INPUT_0, T_OUTPUT_0, T_SEMICOLON_0, T_CONTINUE_WHILE_0, T_EXIT_WHILE_0 }, { "statement_in_while_and_if_body__iteration",{\ {LA_IS, {""}, 2, { "statement_in_while_and_if_body", "statement_in_while_and_if_body__iteration" }}\ }};\ }};\ {LA_NOT, { "ident_terminal", "(", T_NOT_0, "unsigned_value_terminal", T_ADD_0, T_SUB_0, T_IF_0, T_FOR_0, T_WHILE_0, T_REPEAT_0, T_GOTO_0, T_INPUT_0, T_OUTPUT_0, T_SEMICOLON_0, T_CONTINUE_WHILE_0, T_EXIT_WHILE_0 }, { "statement_in_while_and_if_body__iteration",{\ {LA_IS, {""}, 0, { "" }}\ }};\ }};\ |
| while_cycle_head_expression = expression;                                                             |  |  |  | while_cycle_head_expression = SAME_RULE(expression);                                                               |  | {LA_IS, { "(", T_NOT_0, T_ADD_0, T_SUB_0, "ident_terminal", "unsigned_value_terminal", "T_IF_0" }, { "while_cycle_head_expression",{\ {LA_IS, {""}, 1, { "expression" }}\ }};\ }};\                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |
| while_cycle = "WHILE" , while_cycle_head_expression , block_statements_in_while_and_if_body;          |  |  |  | while_cycle = tokenWHILE >> while_cycle_head_expression >> block_statements_in_while_and_if_body;                  |  | {LA_IS, { T_WHILE_0 }, { "while_cycle",{\ {LA_IS, {""}, 3, { T_WHILE_0, "while_cycle_head_expression", "block_statements_in_while_and_if_body" }}\ }};\ }};\                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |
| repeat_until_cycle_cond = expression;                                                                 |  |  |  | repeat_until_cycle_cond = SAME_RULE(expression);                                                                   |  | {LA_IS, { "(", T_NOT_0, T_ADD_0, T_SUB_0, "ident_terminal", "unsigned_value_terminal", "T_IF_0" }, { "repeat_until_cycle_cond",{\ {LA_IS, {""}, 1, { "expression" }}\ }};\ }};\                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
| repeat_until_cycle = "REPEAT" , ({statement}   block_statements) , "UNTIL" , repeat_until_cycle_cond; |  |  |  | repeat_until_cycle = tokenREPEAT >> (*statement   block_statements) >> tokenUNTIL >> repeat_until_cycle_cond;      |  | {LA_IS, { T_REPEAT_0 }, { "repeat_until_cycle",{\ {LA_IS, {""}, 4, { T_REPEAT_0, "statements__or__block_statements", T_UNTIL_0, "repeat_until_cycle_cond" }}\ }};\ }};\                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |
|                                                                                                       |  |  |  |                                                                                                                    |  | {LA_IS, { "ident_terminal", "(", T_NOT_0, "unsigned_value_terminal", T_ADD_0, T_SUB_0, T_IF_0, T_FOR_0, T_WHILE_0, T_REPEAT_0, T_GOTO_0, T_INPUT_0, T_OUTPUT_0, T_SEMICOLON_0 }, { "statements__or__block_statements",{\ {LA_IS, {""}, 1, { "statement__iteration" }}\ }};\ }};\                                                                                                                                                                                                                                                                                                                                                                                                                |

|                                                                                                                                                                                                        |  |  |  |                                                                                                                                                                                                                                               |  |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--|--|--|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                                                                                                                                                                                                        |  |  |  |                                                                                                                                                                                                                                               |  | {LA_IS, {T_BEGIN_BLOCK_0 }, { "statements__or__block_statements",{\n{LA_IS, {""}, 1, { "block_statements" }}}\n}};\n                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |
| input = "GET" , ( ident , [index_action]   "(" , ident , [index_action] , ")" );                                                                                                                       |  |  |  | input = tokenGET >> (ident >> -index_action   tokenGROUPEXPRESSIONBEGIN >> ident >> -index_action >> tokenGROUPEXPRESSIONEND);                                                                                                                |  | {LA_IS, {T_INPUT_0 }, { "input_rule",{\n{LA_IS, {""}, 2, {T_INPUT_0, "argument_for_input" }}\n}};\n                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |
|                                                                                                                                                                                                        |  |  |  |                                                                                                                                                                                                                                               |  | {LA_IS, { "ident_terminal" }, { "argument_for_input",{\n{LA_IS, {""}, 2, { "ident", "index_action__optional" }}\n}};\n{LA_IS, { "(" }, { "argument_for_input",{\n{LA_IS, {""}, 4, { "(", "ident", "index_action__optional", ")" }}\n}};\n                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |
| output = "PUT" , expression;                                                                                                                                                                           |  |  |  | output = tokenPUT >> expression;                                                                                                                                                                                                              |  | {LA_IS, {T_OUTPUT_0 }, { "output_rule", {\n{LA_IS, {""}, 2, {T_OUTPUT_0, "expression" }}\n}};\n                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |
| statement = expression_or_bind_left_to_right   cond_block__with_optional_bind   forto_cycle   while_cycle   repeat_until_cycle   labeled_point   goto_label   input   output   ";"                     |  |  |  | statement = expression_or_bind_left_to_right   cond_block__with_optional_bind   forto_cycle   while_cycle   repeat_until_cycle   labeled_point   goto_label   input   output   tokenSEMICOLON;                                                |  | {LA_IS, { "ident_terminal" }, { "statement", {\n{LA_IS, {T_COLON_0 }, 1, { "labeled_point" }};\n{LA_NOT, {T_COLON_0 }, 1, { "expression_or_bind_left_to_right" }}\n}};\n{LA_IS, { "(" , T_NOT_0, "unsigned_value_terminal", T_ADD_0, T_SUB_0 }, { "statement", {\n{LA_IS, {""}, 1, { "expression_or_bind_left_to_right" }}\n}};\n{LA_IS, {T_IF_0 }, { "statement",{\n{LA_IS, {""}, 1, { "cond_block__with_optional_bind" }}\n}};\n{LA_IS, {T_FOR_0 }, { "statement",{\n{LA_IS, {""}, 1, { "forto_cycle" }}\n}};\n{LA_IS, {T_WHILE_0 }, { "statement",{\n{LA_IS, {""}, 1, { "while_cycle" }}\n}};\n{LA_IS, {T_REPEAT_0 }, { "statement",{\n{LA_IS, {""}, 1, { "repeat_until_cycle" }}\n}};\n{LA_IS, {T_GOTO_0 }, { "statement",{\n{LA_IS, {""}, 1, { "goto_label" }}\n}};\n{LA_IS, {T_INPUT_0 }, { "statement",{\n{LA_IS, {""}, 1, { "input_rule" }}\n}};\n{LA_IS, {T_OUTPUT_0 }, { "statement",{\n{LA_IS, {""}, 1, { "output_rule" }}\n}};\n{LA_IS, {T_SEMICOLON_0 }, { "statement",{\n{LA_IS, {""}, 1, { ";" }}\n}};\n |
|                                                                                                                                                                                                        |  |  |  |                                                                                                                                                                                                                                               |  | {LA_IS, { "ident_terminal", "(", T_NOT_0, "unsigned_value_terminal", T_ADD_0, T_SUB_0, T_IF_0, T_FOR_0, T_WHILE_0, T_REPEAT_0, T_GOTO_0, T_INPUT_0, T_OUTPUT_0, T_SEMICOLON_0 }, { "statement__iteration", {\n{LA_IS, {""}, 2, { "statement", "statement__iteration" }}\n}};\n{LA_NOT, { "ident_terminal", "(", T_NOT_0, "unsigned_value_terminal", T_ADD_0, T_SUB_0, T_IF_0, T_FOR_0, T_WHILE_0, T_REPEAT_0, T_GOTO_0, T_INPUT_0, T_OUTPUT_0, T_SEMICOLON_0 }, { "statement__iteration", {\n{LA_IS, {""}, 0, { "" }}\n}};\n                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |
| block_statements = "{", {statement}, "}"                                                                                                                                                               |  |  |  | block_statements = tokenBEGINBLOCK >> *statement >> tokenENDBLOCK;                                                                                                                                                                            |  | {LA_IS, {T_BEGIN_BLOCK_0 }, { "block_statements", {\n{LA_IS, {""}, 3, {T_BEGIN_BLOCK_0, "statement__iteration", T_END_BLOCK_0 }}\n}};\n                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
|                                                                                                                                                                                                        |  |  |  |                                                                                                                                                                                                                                               |  | {LA_IS, { "(" , T_NOT_0, T_ADD_0, T_SUB_0, "ident_terminal", "unsigned_value_terminal", "T_IF_0" }, { "expression__optional", {\n{LA_IS, {""}, 1, { "expression" }}\n}};\n{LA_NOT, { "(" , T_NOT_0, T_ADD_0, T_SUB_0, "ident_terminal", "unsigned_value_terminal", "T_IF_0" }, { "expression__optional", {\n{LA_IS, {""}, 0, { "" }}\n}};\n                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |
| program = "NAME" , program_name , ",", "BODY" , "DATA" , [declaration] , ";", {statement} , "END";                                                                                                     |  |  |  | program = BOUNDARIES >> tokenNAME >> program_name >> tokenSEMICOLON >> tokenBODY >> tokenDATA >> {-declaration} >> tokenSEMICOLON >> *statement >> tokenEND;                                                                                  |  | {LA_IS, {T_NAME_0 }, { "program_rule", {\n{LA_IS, {""}, 9, {T_NAME_0, "program_name", T_SEMICOLON_0, T_BODY_0, T_DATA_0, "declaration__optional", T_SEMICOLON_0, "statement__iteration", T_END_0 }}\n}};\n                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
|                                                                                                                                                                                                        |  |  |  |                                                                                                                                                                                                                                               |  | {LA_IS, {T_DATA_TYPE_0 }, { "declaration__optional", {\n{LA_IS, {""}, 1, { "declaration" }}\n}};\n{LA_NOT, {T_DATA_TYPE_0 }, { "declaration__optional", {\n{LA_IS, {""}, 0, { "" }}\n}};\n                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
| digit = "0"   "1"   "2"   "3"   "4"   "5"   "6"   "7"   "8"   "9";                                                                                                                                     |  |  |  | digit = digit_0   digit_1   digit_2   digit_3   digit_4   digit_5   digit_6   digit_7   digit_8   digit_9;                                                                                                                                    |  | \                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
| non_zero_digit = "1"   "2"   "3"   "4"   "5"   "6"   "7"   "8"   "9";                                                                                                                                  |  |  |  | non_zero_digit = digit_1   digit_2   digit_3   digit_4   digit_5   digit_6   digit_7   digit_8   digit_9;                                                                                                                                     |  | \                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
| unsigned_value = (non_zero_digit , (digit))   "0";                                                                                                                                                     |  |  |  | unsigned_value = (non_zero_digit >> *digit   digit_0) >> BOUNDARIES;                                                                                                                                                                          |  | \                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
| value = [sign] , unsigned_value;                                                                                                                                                                       |  |  |  | value = -sign >> unsigned_value >> BOUNDARIES;                                                                                                                                                                                                |  | {LA_IS, { "unsigned_value_terminal" }, { "unsigned_value", {\n{LA_IS, {""}, 1, { "unsigned_value_terminal" }}\n}};\n{LA_IS, { "unsigned_value_terminal", T_ADD_0, T_SUB_0 }, { "value", {\n{LA_IS, {""}, 2, { "sign_optional", "unsigned_value" }}\n}};\n                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |
|                                                                                                                                                                                                        |  |  |  |                                                                                                                                                                                                                                               |  | {LA_IS, {T_ADD_0, T_SUB_0 }, { "sign_optional", {\n{LA_IS, {""}, 1, { "sign" }}\n}};\n{LA_NOT, {T_ADD_0, T_SUB_0 }, { "sign_optional", {\n{LA_IS, {""}, 0, { "" }}\n}};\n                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |
| letter_in_lower_case = "a"   "b"   "c"   "d"   "e"   "f"   "g"   "h"   "i"   "j"   "k"   "l"   "m"   "n"   "o"   "p"   "q"   "r"   "s"   "t"   "u"   "v"   "w"   "x"   "y"   "z";                      |  |  |  | letter_in_lower_case = a   b   c   d   e   f   g   h   i   j   k   l   m   n   o   p   q   r   s   t   u   v   w   x   y   z;                                                                                                                 |  | \                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
| letter_in_upper_case = "A"   "B"   "C"   "D"   "E"   "F"   "G"   "H"   "I"   "J"   "K"   "L"   "M"   "N"   "O"   "P"   "Q"   "R"   "S"   "T"   "U"   "V"   "W"   "X"   "Y"   "Z";                      |  |  |  | letter_in_upper_case = A   B   C   D   E   F   G   H   I   J   K   L   M   N   O   P   Q   R   S   T   U   V   W   X   Y   Z;                                                                                                                 |  | \                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
| ident = "._" , letter_in_upper_case , letter_in_upper_case , letter_in_upper_case , letter_in_upper_case , letter_in_upper_case , letter_in_upper_case , letter_in_upper_case , letter_in_upper_case ; |  |  |  | ident = tokenUNDERSCORE >> letter_in_upper_case >> letter_in_upper_case >> letter_in_upper_case >> letter_in_upper_case >> letter_in_upper_case >> letter_in_upper_case >> letter_in_upper_case >> letter_in_upper_case >> STRICT_BOUNDARIES; |  | {LA_IS, { "ident_terminal" }, { "ident", {\n{LA_IS, {""}, 1, { "ident_terminal" }}\n}};\n                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |
| sign = "+"   "-";                                                                                                                                                                                      |  |  |  | sign = sign_plus   sign_minus;                                                                                                                                                                                                                |  | {LA_IS, {T_ADD_0 }, { "sign", {\n{LA_IS, {""}, 1, { "sign_plus" }}\n}};\n                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |

[illegible]

|  |  |  |  |                                                                                                           |  |   |
|--|--|--|--|-----------------------------------------------------------------------------------------------------------|--|---|
|  |  |  |  |                                                                                                           |  | \ |
|  |  |  |  |                                                                                                           |  | \ |
|  |  |  |  |                                                                                                           |  | \ |
|  |  |  |  |                                                                                                           |  | \ |
|  |  |  |  |                                                                                                           |  | \ |
|  |  |  |  | STRICT_BOUNDARIES = (BOUNDARY >> *(BOUNDARY))   (!(qi::alpha   qi::char_("_ ")));                         |  | \ |
|  |  |  |  | BOUNDARIES = (BOUNDARY >> *(BOUNDARY)   NO_BOUNDARY);                                                     |  | \ |
|  |  |  |  | BOUNDARY = BOUNDARY_SPACE   BOUNDARY_TAB   BOUNDARY_CARRIAGE_RETURN   BOUNDARY_LINE_FEED   BOUNDARY_NULL; |  | \ |
|  |  |  |  | BOUNDARY_SPACE = " ";                                                                                     |  | \ |
|  |  |  |  | BOUNDARY_TAB = "\t";                                                                                      |  | \ |
|  |  |  |  | BOUNDARY_CARRIAGE_RETURN = "\r";                                                                          |  | \ |
|  |  |  |  | BOUNDARY_LINE_FEED = "\n";                                                                                |  | \ |
|  |  |  |  | BOUNDARY_NULL = "\0";                                                                                     |  | \ |
|  |  |  |  | NO_BOUNDARY = "";                                                                                         |  | \ |
|  |  |  |  | tokenUNDERSCORE = "_";                                                                                    |  | \ |
|  |  |  |  | A = "A";                                                                                                  |  | \ |
|  |  |  |  | B = "B";                                                                                                  |  | \ |
|  |  |  |  | C = "C";                                                                                                  |  | \ |
|  |  |  |  | D = "D";                                                                                                  |  | \ |
|  |  |  |  | E = "E";                                                                                                  |  | \ |
|  |  |  |  | F = "F";                                                                                                  |  | \ |
|  |  |  |  | G = "G";                                                                                                  |  | \ |
|  |  |  |  | H = "H";                                                                                                  |  | \ |
|  |  |  |  | I = "I";                                                                                                  |  | \ |
|  |  |  |  | J = "J";                                                                                                  |  | \ |
|  |  |  |  | K = "K";                                                                                                  |  | \ |
|  |  |  |  | L = "L";                                                                                                  |  | \ |
|  |  |  |  | M = "M";                                                                                                  |  | \ |
|  |  |  |  | N = "N";                                                                                                  |  | \ |
|  |  |  |  | O = "O";                                                                                                  |  | \ |
|  |  |  |  | P = "P";                                                                                                  |  | \ |
|  |  |  |  | Q = "Q";                                                                                                  |  | \ |
|  |  |  |  | R = "R";                                                                                                  |  | \ |
|  |  |  |  | S = "S";                                                                                                  |  | \ |
|  |  |  |  | T = "T";                                                                                                  |  | \ |
|  |  |  |  | U = "U";                                                                                                  |  | \ |
|  |  |  |  | V = "V";                                                                                                  |  | \ |
|  |  |  |  | W = "W";                                                                                                  |  | \ |
|  |  |  |  | X = "X";                                                                                                  |  | \ |
|  |  |  |  | Y = "Y";                                                                                                  |  | \ |
|  |  |  |  | Z = "Z";                                                                                                  |  | \ |
|  |  |  |  | a = "a";                                                                                                  |  | \ |
|  |  |  |  | b = "b";                                                                                                  |  | \ |
|  |  |  |  | c = "c";                                                                                                  |  | \ |
|  |  |  |  | d = "d";                                                                                                  |  | \ |
|  |  |  |  | e = "e";                                                                                                  |  | \ |
|  |  |  |  | f = "f";                                                                                                  |  | \ |
|  |  |  |  | g = "g";                                                                                                  |  | \ |
|  |  |  |  | h = "h";                                                                                                  |  | \ |
|  |  |  |  | i = "i";                                                                                                  |  | \ |
|  |  |  |  | j = "j";                                                                                                  |  | \ |
|  |  |  |  | k = "k";                                                                                                  |  | \ |
|  |  |  |  | l = "l";                                                                                                  |  | \ |
|  |  |  |  | m = "m";                                                                                                  |  | \ |
|  |  |  |  | n = "n";                                                                                                  |  | \ |
|  |  |  |  | o = "o";                                                                                                  |  | \ |

|  |  |  |  |          |  |                                                                        |
|--|--|--|--|----------|--|------------------------------------------------------------------------|
|  |  |  |  | p = "p"; |  | \                                                                      |
|  |  |  |  | q = "q"; |  | \                                                                      |
|  |  |  |  | r = "r"; |  | \                                                                      |
|  |  |  |  | s = "s"; |  | \                                                                      |
|  |  |  |  | t = "t"; |  | \                                                                      |
|  |  |  |  | u = "u"; |  | \                                                                      |
|  |  |  |  | v = "v"; |  | \                                                                      |
|  |  |  |  | w = "w"; |  | \                                                                      |
|  |  |  |  | x = "x"; |  | \                                                                      |
|  |  |  |  | y = "y"; |  | \                                                                      |
|  |  |  |  | z = "z"; |  | \                                                                      |
|  |  |  |  |          |  | \                                                                      |
|  |  |  |  |          |  | \                                                                      |
|  |  |  |  |          |  | \                                                                      |
|  |  |  |  |          |  | { LA_IS, { T_NAME_0 }, { "program____part1", {\                        |
|  |  |  |  |          |  | { LA_IS, {} }, 7, { T_NAME_0, "program_name", T_SEMICOLON_0, T_BODY_0, |
|  |  |  |  |          |  | T_DATA_0, "declaration_optional", T_SEMICOLON_0 }}\                    |
|  |  |  |  |          |  | }});\                                                                  |
|  |  |  |  |          |  | \                                                                      |
|  |  |  |  |          |  | );\                                                                    |
|  |  |  |  |          |  | 103,\                                                                  |
|  |  |  |  |          |  | "program_rule"                                                         |